

PLANDEX_ANALYSIS

Plandex - Comprehensive Analysis for HelixCode Porting

Executive Summary

Plandex is a sophisticated AI coding agent designed for large-scale development tasks. It's built in Go (server) and TypeScript/Go (CLI) with a terminal-first interface. The system combines intelligent context management, multi-provider LLM integration, file diff review, git integration, and progressive autonomy levels.

1. Core Features and Capabilities

1.1 Plans & Workflow Management

- **Plans:** Conversation-like containers for tasks (similar to ChatGPT sessions)
 - Can contain single prompts or complex multi-step workflows
 - Support for context (files, directories, URLs, images, notes, piped data)
 - Each plan maintains independent conversation history and pending changes
 - Plans are project-directory scoped with support for subdirectories
 - Plans can be named, archived, or deleted

1.2 Conversation System

- **Chat Mode** (default): Flex out ideas before implementation
- **Tell Mode:** Detailed planning and code generation
- Conversation history with version-controlled entries
- Automatic conversation summarization via “summarizer” role
- Git diff format viewing of conversation changes
- Full conversation export via convo command

1.3 Context Management

- **Dual approach:** Manual + Automatic context loading
- **Project Maps:** Tree-sitter based syntax trees for 30+ languages (~100k tokens per file)
- **File loading:** Load specific files or entire directories

- **Smart Context Window:**
 - Dynamically loads only relevant files per task step
 - Reduces context bloat in large projects
 - Works with both automatic and manual loading
- **Auto-context fallback:** Falls back to simpler models when context exceeds limits
- **Context Versioning:** All context changes tracked in plan history

1.4 Change Management & Diff Review

- **Sandbox model:** Changes kept separate from project until approved
- **Cumulative diff sandbox:** All changes accumulate until explicitly applied
- **Diff viewing:**
 - Terminal: `git diff` format
 - UI: Browser-based diff viewer with side-by-side and line-by-line views
 - Can reject individual files without affecting others
- **Change staging:** Changes can be applied selectively
- **Git integration:** Auto-commit with AI-generated commit messages

1.5 Autonomy System (5 Levels)

Progressive automation from manual to full: - **None:** Complete manual control - **Basic:** Auto-continue plans + auto-build (v1 equivalent) - **Plus:** Smart context + manual execution + auto-commit - **Semi** (default): Auto-load context + manual apply - **Full:** Complete automation with auto-debug and rollback

Each level configurable individually via `set-config` commands.

1.6 Execution & Debugging

- **Command execution:** Run build/test/deployment commands
- **Auto-debugging:** Terminal and browser-based debugging
- **Chrome integration:** Auto-debug for browser applications
- **Rollback:** Failed executions automatically rollback changes
- **Execution status tracking:** AI determines if plan should continue based on results

1.7 Version Control System

- **Plan versioning:** Every action creates a version
 - Context changes, prompts, responses, builds, applications
- **Branching:** Create branches for exploring alternatives
- **Rewind capability:** Revert to earlier plan states
- **Git hooks:** Integration with project git for commit/push automation
- **History viewing:** Full plan dex log with optional revert

1.8 Role-Based Model System

Specialized roles for different tasks: - **planner**: Main role, replies to prompts and makes plans - **architect**: High-level planning + context selection (optional, falls back to planner) - **coder**: Code implementation with strict formatting rules (optional) - **builder**: Builds plans into file diffs - **whole-file-builder**: File rewrite fallback when targeted edits fail - **summarizer**: Conversation summarization - **auto-continue**: Determines plan completion - **names**: Auto-naming for plans and context - **commit-messages**: Generates git commit messages

2. Supported LLM Providers and APIs

2.1 Built-in Providers (12 total)

1. **OpenAI** (ModelProviderOpenAI)
 - Direct API access
 - Environment: OPENAI_API_KEY, OPENAI_ORG_ID (optional)
 - Base URL: <https://api.openai.com/v1>
2. **Anthropic** (ModelProviderAnthropic)
 - Direct Claude API
 - Environment: ANTHROPIC_API_KEY
 - Special: Claude Max subscription support (OAuth tokens)
 - Beta header: anthropic-beta: oauth-2025-04-20
3. **Anthropic Claude Max** (ModelProviderAnthropicClaudeMax)
 - Enhanced Claude Max with extended context
 - Environment: Separate Pro/Max credentials
4. **Google AI Studio** (ModelProviderGoogleAISTudio)
 - Gemini models (free tier)
 - Environment: GEMINI_API_KEY
5. **Google Vertex AI** (ModelProviderGoogleVertex)
 - Enterprise Gemini/Anthropic models
 - Environment: GOOGLE_APPLICATION_CREDENTIALS, VERTEXAI_PROJECT, VERTEXAI_LOCATION
6. **Azure OpenAI** (ModelProviderAzureOpenAI)
 - OpenAI models via Azure infrastructure
 - Environment: AZURE_OPENAI_API_KEY, AZURE_API_BASE, AZURE_API_VERSION
 - Optional: AZURE_DEPLOYMENTS_MAP for custom deployment names
7. **AWS Bedrock** (ModelProviderAmazonBedrock)
 - Anthropic models via AWS
 - Environment: AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_REGION
 - Optional: AWS_SESSION_TOKEN, AWS_INFERENCE_PROFILE_ARN

- Also supports: PLANDEX_AWS_PROFILE for credential file
- 8. **OpenRouter** (ModelProviderOpenRouter)
 - Aggregator supporting 100+ models
 - Environment: OPENROUTER_API_KEY
 - Base URL: `https://openrouter.ai/api/v1`
 - **Special Role:** Automatic failover for other providers
- 9. **DeepSeek** (ModelProviderDeepSeek)
 - Chinese reasoning models
 - Environment: DEEPSEEK_API_KEY
- 10. **Perplexity** (ModelProviderPerplexity)
 - Web-aware reasoning models
 - Environment: PERPLEXITY_API_KEY
- 11. **Ollama** (ModelProviderOllama)
 - Local models
 - No authentication required
 - Local only
 - Environment: OLLAMA_BASE_URL
- 12. **Custom** (ModelProviderCustom)
 - OpenAI-compatible endpoints
 - Self-hosted LLM servers
 - Configurable via JSON config file

2.2 Provider Selection Logic

- **Priority order:** Direct provider > Aggregator (OpenRouter) > Fallback
- **Environment variable detection:** Checks which credentials are set at runtime
- **Direct provider preference:** If both direct and OpenRouter are available, uses direct
- **OpenRouter failover:** Falls back to OpenRouter if primary provider fails
- **LiteLLM proxy integration:** Python layer handles provider routing
 - Special OAuth handling for Anthropic
 - Supports 4+ AWS credential patterns

2.3 Custom Provider Support

- OpenAI-compatible API requirement
 - JSON configuration: `/models custom` REPL command
 - Self-hosting compatible
 - Not available on Plandex Cloud (Integrated Models mode)
-

3. Supported Models

3.1 Model Metadata Structure

Each model has: - **ModelTag**: Unique identifier (e.g., “openai/gpt-4.1”) - **ModelId**: Internal ID for disambiguation - **Publisher**: OpenAI, Anthropic, Google, DeepSeek, Perplexity, Qwen, Mistral - **MaxTokens**: Absolute input limit (200k, 400k, 1M+) - **MaxOutputTokens**: Output capacity (8k-100k) - **ReservedOutputTokens**: Context window reservation (8k-40k) - **DefaultMaxConvoTokens**: Before summarization triggers - **PreferredOutputFormat**: Either JSON (OpenAI) or XML (others) - **SystemPromptDisabled**: For early OpenAI releases - **RoleParamsDisabled**: When model doesn’t support temperature, top_p, etc. - **SupportsCacheControl**: For prompt caching (Anthropic, OpenAI, Google)

3.2 Current Model Portfolio (25+ models across 7 publishers)

OpenAI Models: - o3, o4-mini (reasoning models with variants: high/medium/low) - gpt-4.1 (1M+ context window) - gpt-4o, gpt-4o-mini - gpt-4-turbo

Anthropic Models: - Claude 3.5 Sonnet (3.5 Sonnet, 3 Opus variants) - Claude 3 Haiku - Extended context versions (200k tokens)

Google Models: - Gemini 2.0 Flash (1M token context) - Gemini 1.5 Pro/Flash - Gemini 1.5 Flash Exp

DeepSeek: - DeepSeek R1 (reasoning model) - DeepSeek V3

Perplexity: - Sonar (web-aware reasoning)

Local/Open Source: - Mistral variants - Qwen models - Llama models (via Ollama)

3.3 Model Variants

Some models have multiple variants with different parameter tuning: -

Reasoning models (o3, o4-mini, R1): high/medium/low reasoning effort variants - Variants override base model settings (reserved tokens, reasoning budget) - **Large context fallbacks**: Alternative model when context exceeds limits

3.4 Built-in Model Packs (16 total)

Pre-configured combinations for different use cases:

1. **DailyDriver** (Default): Balanced cost/performance
 - Multi-provider mix (Anthropic, OpenAI, Google)
2. **Reasoning**: Best for complex logic
 - o3-mini planner + specialized models

3. **Strong**: Maximum capability
 - High-end models across all roles
 4. **Cheap**: Cost-optimized
 - Budget-friendly models
 5. **OSS** (Open Source): Local models via Ollama
 6. **AnthropicOnly**: Exclusive Anthropic usage
 7. **OpenAIOnly**: Exclusive OpenAI usage
 8. **GoogleOnly**: Exclusive Google usage
 9. **GeminiPlanner**: Gemini-focused planning
 10. **OpusPlanner**: Anthropic Opus planning
 11. **O3Planner**: OpenAI o3 planning
 12. **R1Planner**: DeepSeek R1 planning
 13. **PerplexityPlanner**: Perplexity-focused
 14. **OllamaExperimental**: Experimental local models
 15. **OllamaAdaptiveOSS**: Auto-selecting local models
 16. **OllamaAdaptiveDaily**: Hybrid cloud+local
-

4. Key Technical Implementations

4.1 Architecture Overview

Server: Go (Gorilla Mux HTTP framework) - Port: 8099 (default), 4000 (LiteLLM proxy) - Database: PostgreSQL (8000+ lines of schema) - Language support: 30+ via tree-sitter

CLI: Go + TypeScript - Terminal UI with fuzzy autocomplete - REPL mode with stateful context - Streaming response handling

LLM Integration: Python LiteLLM proxy - Abstraction layer for all providers - OAuth header handling for Anthropic - Streaming response support - Error handling and retries

4.2 Database Schema (8351 lines of Go code)

Major entities: - plans: Core plan data with status, config, autonomy settings - plan_versions: Complete history of all changes - contexts: File/directory/URL/image context entries - context_versions: Versioned context changes - messages: Conversation messages with streaming data - pending_changes: Staged file modifications (replacements) - branches: Plan branches for exploration - git_history: Commit integration tracking

Helpers for: - Context management (load, update, remove, map) - Plan operations (CRUD, execution, branching) - Result handling (replacements, pending summary) - Queue management (background tasks) - RBAC (role-based access control)

4.3 File Editing System

Structured Edits (Primary approach): - Tree-sitter based AST-aware edits - Language-specific parsing for 30+ languages - Section-based editing (functions, classes, etc.) - Generic fallback patterns - Validation and retry with whole-file fallback

File Management: - Diff generation via `git diff` (no-index mode) - Replacement tracking with UUIDs - Syntax validation before applying - Multi-file transaction support

Fallback Chain: 1. Structured edits (targeted changes) 2. Validate & fix (syntax correction) 3. Whole-file builder role (complete rewrite) 4. Builder role with different approach

4.4 Context Management

Project Mapping: - Tree-sitter based syntax tree generation - File map supports 30+ languages - CLI-based (`cli/fs`, `cli/lib`) file discovery - Caching of project structure - Support for 20M+ token projects

Context Loading Strategy: - Architect role selects relevant files from project map - Smart window filtering loads only files for current step - Cache control directives for prompt caching - Separate pipelines for auto vs. manual loading

Context Caching: - Anthropic, OpenAI, Google support `SupportsCacheControl` - Cache savings tracked in response metadata - Per-path caching flags (`CachedByPath`) - Cost optimization via caching

4.5 Streaming & Real-time Updates

Response Streaming: - Server-side streaming with chunked responses - Plan execution streaming (`tell_stream_*`) - Error streaming with auto-recovery - Status/progress updates during execution

WebSocket Integration: - Real-time plan updates - Connection pooling - Message queueing

4.6 Error Handling & Resilience

Multi-level Recovery: 1. Provider-specific error handling 2. Automatic fallback to alternative provider 3. Syntax validation and correction 4. Whole-file rebuild as last resort 5. Manual user intervention options

Execution Rollback: - Command execution failures trigger file rollback - Git state preservation for manual recovery - Conflict detection when rewinding

4.7 Git Integration

Features: - Automatic git status checking - Commit message generation via AI - Optional auto-commit on successful apply - Revert capability matching plan versions - Branch-aware operations

Files: db/git.go, plan/commit_msg.go

5. Unique Features

5.1 Cumulative Diff Review Sandbox

- **Distinctive:** Most similar tools apply changes immediately
- **Plandex approach:** All changes accumulate in version-controlled sandbox
- **Benefits:**
 - Review entire feature before applying
 - Selective file rejection
 - Complete rollback capability
 - Browser-based UI for complex diffs

5.2 Progressive Autonomy with Fallbacks

- **Level-based preset config:** From manual to full automation
- **Fallback chain:** Large context → Error → Use stronger model
- **Runtime reconfigurable:** Change during plan execution
- **Safety features:** Config warnings for full auto mode

5.3 Multi-role Model System

- **Specialization:** Different models for planning, coding, building
- **Fallback support:** Each role has optional fallback
- **Role-specific tuning:**
 - Planner: Large context, good reasoning
 - Coder: Strong instruction following
 - Builder: Precise formatting required
 - Auto-continue: Classification task optimization

5.4 Smart Context Window Management

- **Dynamic sizing:** Grows/shrinks per task step
- **Relevance filtering:** Only loads files for current step
- **Works with both:** Automatic and manual context loading
- **Solves n-file problem:** Doesn't reload all 10 files when editing 1

5.5 Tree-sitter Based Code Editing

- **Language support:** 30+ languages out of the box
- **Syntax-aware edits:** Understands structure, not just text
- **AST precision:** Targets specific functions/classes/blocks
- **Fallback robustness:** Reverts to whole-file if targeted edits fail

5.6 Full Plan Versioning

- **Granular history:** Every action versioned
- **Reversible operations:** Rewind with conflict detection
- **Branch capability:** Explore alternatives without losing history
- **Conversation preservation:** Full message history accessible

5.7 Context Caching for Cost/Speed

- **Provider-native support:** Leverages built-in caching
- **Automatic optimization:** Detects cacheable patterns
- **Cost tracking:** Reports cache savings
- **Works across:** Anthropic, OpenAI, Google

5.8 Browser-based Debugging

- **Visual inspection:** Watch the code execute
- **Screenshot capture:** Auto-captures browser state on failure
- **Auto-detect issues:** Finds visual errors beyond test failures
- **Integrated debugging:** Feeds screenshots back to model

5.9 Integrated Version Control

- **No separate git needed:** Works without git repo
- **Optional git integration:** Auto-commits if repo exists
- **Message generation:** AI writes commit messages
- **Branch support:** Git branches for alternative approaches

6. Key Technical Patterns for Porting to HelixCode

6.1 LLM Integration Pattern

Provider Selection → LiteLLM Proxy → Direct API
↓
Fallback Chain

↓

Auto-retry with another provider

Actionable: HelixCode should maintain a provider fallback chain and support multiple simultaneous providers.

6.2 Role-based Model Routing

- Each workflow step specifies required role
- System resolves role → model → provider at runtime
- Supports fallback to parent role if specific role not configured
- Enables mixing models without explicit chain

Actionable: Implement task-role mapping separate from model selection.

6.3 Context Management Stages

1. Discovery (project map generation)
2. Selection (architect role picks relevant files)
3. Filtering (smart context loads only current step)
4. Caching (prompt cache directives applied)

Actionable: Design context pipeline as composable stages.

6.4 Multi-model Output Formats

- OpenAI models prefer JSON (strict schema support)
- Other providers work better with XML
- Dynamically set PreferredOutputFormat per model
- Parser handles both transparently

Actionable: Detect model capabilities and adapt output format parsing.

6.5 Streaming Response Architecture

- Chunk-based streaming with type prefixes
- Separate goroutines for streaming and storage
- Error recovery during stream
- Progress indicators between chunks

Actionable: Use typed message framing for stream disambiguation.

6.6 File Edit Validation

Proposed Edit

↓

Try Structured Edit (tree-sitter)
↓ (if fails)
Syntax Validation & Fix
↓ (if fails)
Try Whole-File Builder
↓ (if fails)
Try Alternative Approach
↓
Manual User Fix

Actionable: Build edit validation with pluggable fallback strategies.

6.7 Autonomy Configuration

- Preset levels (none, basic, plus, semi, full)
- Individual flags can override level
- Runtime reconfigurable
- Each feature checks autonomy before proceeding

Actionable: Use hierarchical config with preset templates but individual flag overrides.

6.8 Plan Version Control

- Minimal version metadata (action, timestamp, delta summary)
- Full content in separate tables
- Rewind creates new branch implicitly
- Conflict detection on revert

Actionable: Store plan metadata separately from deltas for efficient history.

6.9 Diff Generation

- Git-based: Works on any file, no language-specific knowledge needed
- Hunk parsing: Structured parsing of unified diff format
- Replacement tracking: UUIDs for matching proposed ↔ actual
- Selective application: Individual file acceptance/rejection

Actionable: Use git diff as lingua franca for all file comparisons.

6.10 Provider Credential Management

- Environment variables per provider (not config files)
- Lazy credential loading (only when provider used)
- OAuth token support (Anthropic example)
- AWS credential chain support (profile + env vars)

Actionable: Implement env-var-first credential discovery with profile fallback.

7. Database & Infrastructure

7.1 PostgreSQL Schema

- 50+ migration files (fully versioned)
- RBAC support built-in
- Org-based multi-tenancy ready
- Message streaming columns for real-time updates

7.2 Containerization

- Single Dockerfile with Python+Go (for LiteLLM proxy)
- Docker Compose for local development
- Volume mounts for persistent data
- Network isolation between services

7.3 Configuration

- Environment variable driven
 - Runtime database connection
 - Org/user settings in database
 - No local config files in production
-

8. Porting Recommendations for HelixCode

Must-Have Features

1. **Multi-provider LLM integration** with fallback chain
2. **Context management** with smart filtering
3. **File diff review** before applying changes
4. **Role-based model routing** for specialized tasks
5. **Progressive autonomy levels** with individual flag overrides
6. **Plan version control** with branching and rewind

Nice-to-Have Features

1. Tree-sitter based AST-aware editing
2. Browser debugging integration
3. Project map generation for large codebases
4. Conversation summarization

5. Git integration with auto-commits
6. Web-based diff UI

Architectural Patterns to Adopt

1. LiteLLM proxy pattern for provider abstraction
2. Role-based model configuration
3. Staged context management pipeline
4. Fallback chain error handling
5. Minimal plan metadata with separate deltas
6. Environment variable credential discovery
7. Stream-based response handling with type framing

Porting Effort Estimate

- **Provider integration:** 2-3 weeks (framework setup already exists in HelixCode)
 - **Context management:** 2-3 weeks (new system, complex)
 - **Diff review + versioning:** 1-2 weeks (database ready, UI exists)
 - **Role-based routing:** 1 week (model abstraction exists)
 - **Autonomy system:** 1-2 weeks (feature flag system needed)
 - **Total:** ~8-12 weeks for MVP with core features
-

References

- Plandex GitHub: <https://github.com/plandex-ai/plandex>
- Documentation: <https://docs.plandex.ai/>
- Local Mode Quickstart: <https://docs.plandex.ai/hosting/self-hosting/local-mode-quickstart>
- v2.0.0 Release: Autonomy levels and context management overhaul